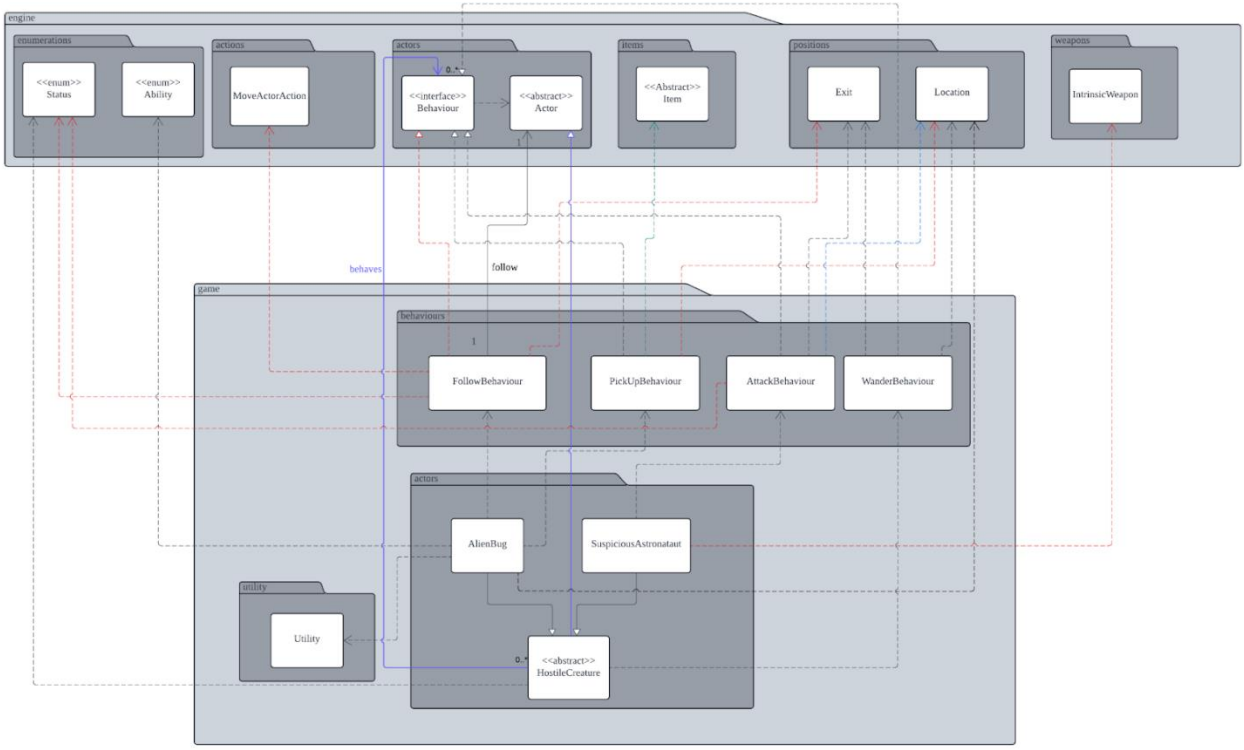
Assignment 2 Design Rationale

Requirement 2: The imposter among us

UML Diagram:



Design Goal:

The design goal for this requirement is mainly to make the Alien Bug wander around the map, capable of picking up scraps found on where they are standing, follow the intern if Intern is within surroundings of bugs, drops all item when dead, and make Suspicious Astronaut wander around and instantly kill the Intern no matter the health with 100% hit rate when Intern is within the surrounding of Suspicious Astronaut. All this is done while adhering closely to relevant design principles and best practices.

Design Decision:

In implementing the ability for alien bug to wander, picking up scraps and follow the intern, the decision was to create separate classes, which is WanderBehaviour class, PickUpBehaviour class, this allows for easier maintainability and easier extendibility since if we need to change how a behaviour works, we will just need to modify one class. Besides, we override the unconscious method to let the bug to drop all items when they die. In addition, we put WanderBehaviour in HostileCreature abstract class so that we do not need to write additional lines of code in subclasses to add this behavious since all creatures can wander. Moreover, we override the getIntrinsicWeapon() in Actor for Suspicious Astronaut to allow it to have intrinsic weapon that instantly kill the actor.

Alternative Design:
One alternative approach to make each creature have their own behaviours is to have one behaviour GOD class for each creature. For example, alien bug would have one behaviour GOD class that handles the behaviour of wandering, picking up scraps, and following the intern. This GOD class will then have if-else statement which handles the priority of the behaviours such as the first if is to check whether any item can be pick up now on the ground, second if is to check whether can follow intern now instead of wandering, and lastly is to let alien bug wander around if none of the above can be performed. This similar approach could be done to Suspicious Astronaut as well. However, this would violate **Single Responsibility Principle (SRP)** as well as **Open-Closed Principle (OCP)**. Besides, we can put all WanderBehaviour in each subclasses instead in HostileCreature class. Furthermore, we can create another Action class just for Suspicious Astronaut to attack the intern and make intern die(using unconscious() method) instantly when this action is performed. However, these two implementations would violate **Don't Repeat Yourself (DRY**) principle.

Analysis of Alternative Design:
The alternative design is not ideal because it violates various DRY and SOLID principles:

1. **Single Responsibility Principle (SRP):**
    - A behaviour GOD class for alien bug is created and handles too much logic in a class. It would handle the wandering behaviour logic, pickup scraps logic as well as following intern logic in a class using if else statement. These would make the class having multiple responsibilities.
    - More and more behaviours may be added which makes it harder to maintain and violate **SRP** since one class should have only one responsibility instead of multiple responsibility.
2. **Open-Closed Principle (OCP):**
    - **OCP** states that each software entity should be only open for extension but closed for modification.
    - Since a behaviour GOD class for each creature would be created, if there is a need to add a new behaviour to the creature, the behaviour GOD class would need to be modify by making use of more if else statement. Even worse, if a certain behaviour needed to change priority or the new behaviour has a higher priority, the whole code needs to be refactored and arranged to prioritise each behaviour accordingly which violates **OCP.**
3. **Don't Repeat Yourself (DRY):**
    - WanderBehaviour are occurring in all subclasses where instead, it could be in superclass(HostileCreature) since all creatures can wander around. This violate **DRY principle.**

- Making a new Action class specifically for Suspicious Astronaut to instantly kill the intern by making the intern die by calling unconscious() method in the new class would be redundant since AttackAction is already provided and we can just override the getIntrinsicWeapon for Suspicious Astronaut to have a damage of max value so that when used in AttackAction, it would kill the intern instantly.
- If we make a new Action class specifically for Suspicious Astronaut, we are not making use of our available code and repeating unnecessary code and logic to perform the same action which violate **DRY.**

Final Design:

In our design, we closely adhere to SOLID principles

1. **Single Responsibility Principle:**
    - **Application:** We created different behaviours class for the creature instead of creating a GOD class which handles all behaviours logic for a creature.
    - **Why:** This would adhere to **SRP** principle as each behaviour class now has only one responsibility that it needs to fulfil. For example, WanderBehaviour only handles the logic for wandering around and PickUpBehaviour only handles the logic for picking up items on the ground. There is not more than one responsibility in each behaviour class. This would make it easier to modify if needed and easier to maintain the system.
    - **Pros:** Adhere to **SRP** principle which allows easier modification if needed since if I were required to change a behaviour, I just need to change that one class instead of changing the whole GOD class which may need to refactor a lot of the code.
    - **Cons:** Many classes needed to be created which would cause overcomplication as there are many different behaviours classes to keep in mind and understand for each creature.
2. **Open-Closed Principle (OCP):**
    - **Application:** We created separate class for each behaviour such as FollowBehaviour, WanderBehaviour and PickUpBehaviour. We only create new classes for behaviours instead of modifying existing class to create that behaviour.
    - **Why:** For example, if we were to create a DanceBehaviour class, there would be no change to other class since other behaviour class are not related to DanceBehaviour, which is a different approach to what I mentioned in alternative design section that need more if else statement if more behaviours were to be added. This would help for easier extensibility of the

system as adding new behaviour only involves creating new class instead of changing existing classes.

- **Pros:** Do not need to modify existing classes which makes it easier to extend and maintain the system.
- **Cons:** Similarly, more classes would be added which might takes new developer to understand and familiarize with the classes.

3. **Don't Repeat Yourself (DRY):**
    - **Application:** Put WanderBehaviour in HostileCreature abstract class(parent) instead of its child classes so that we do not repeatedly putting this behaviour in all child classes since all creatures can wander around. Besides, overriding the getIntrinsicWeapon() in Actor for Suspicious Astronaut to used in AttackAction to instantly kill the intern if within its surrounding. This approach is better than creating a new Action class since we could reuse the AttackAction class provided and use it in conjunction with intrinsic weapon of the Suspicious Astronaut to have Integer.MAX_VALUE of damage. Therefore, we don't need to have redundant and repeated code for this implementation.
    - **Why:** This would avoid repetition of code, so it is easier to maintain.
    - **Pros:** No unnecessary repetition of code, easier for understanding and readability.
    - **Cons:** If there is any HostileCreature that does not wander around, then the code needs to be refactored.

Conclusion:

All in all, our chosen design provides a framework that fulfil the requirements stated by carefully considering all **SOLID and DRY** principles, maintainability and extendibility of the system.